Writing skills *for Claude Code* agents.

Technical documentation and authoring guide

Based on the QA agent pipeline in agent-demo-app (automation/.claude)

Local Content Review Platform · Azure DevOps project DemoApp

Version 1.0 · October 2026

centric

Contents

1. Introduction 4

1.1 Audience 4

1.2 What this document covers 4

1.3 Terminology 4

2. The .claude folder 5

2.1 Location 5

2.2 Directory tree 5

2.3 agents/ 6

2.4 skills/ 6

2.5 Supporting files outside .claude 6

2.6 The QA agent pipeline 6

3. Agents and skills 8

3.1 Anatomy of an agent file 8

3.2 Agent or skill? 9

4. How to write a skill 10

4.1 The common skeleton 10

4.2 Frontmatter and description 10

4.3 Role and system context 11

4.4 References, scripts and progressive disclosure 11

4.5 Inputs 11

4.6 Tools and MCP servers 12

4.7 Workflow 12

4.8 Guardrails 13

4.9 Hand-offs between agents 13

4.10 Lessons 13

4.11 Example 14

4.12 Template 14

4.13 Checklist 15

5. Adding MCP servers 16

5.1 Project servers in .mcp.json 16

5.2 The claude mcp CLI 17

5.3 Scopes 17

5.4 Verifying a server 17

6. Calling an agent 18

6.1 Syntax 18

6.2 Related commands 18

7. Permissions 19

7.1 How this repository does it today 19

7.2 Rules in settings.json 20

7.3 Where settings live 20

8. Recommendations 21

Appendix A. Agent quick reference 22

1. Introduction

How to write skills for Claude Code agents, how they are organised in this repository, and how to wire them to MCP servers and permissions.

This document is the authoring guide for the AI agents that support QA on the **Local Content Review Platform**. It is derived from the five agents and skills that already live in `automation/.claude` (`qa-planner`, `qa-explorer`, `qa-test-case-writer`, `qa-bug-reporter` and `qa-automator`). The patterns they share were extracted and written down here, so that new skills follow the same conventions.

1.1 Audience

- QA engineers who write or maintain agents and skills in `automation/.claude`.
- Developers who run the agents against a PBI in Azure DevOps (org `demo-app-gabi`, project `DemoApp`).
- Anyone configuring MCP servers or permissions for Claude Code in this repository.

1.2 What this document covers

Chapter	Content
2. The .claude folder	Directory layout, supporting files and the QA agent pipeline
3. Agents and skills	Anatomy of an agent file and how agents differ from skills
4. How to write a skill	The common SKILL.md skeleton, section by section, with a template and a checklist
5. Adding MCP servers	`.mcp.json`, the `claude mcp` CLI, scopes and verification
6. Calling an agent	`@agent-<name> <task>` and the related commands
7. Permissions	How this repository restricts files and MCP tools today, and `settings.json` rules
8. Recommendations	Gaps found while writing this guide
Appendix A	Quick reference for the five QA agents

1.3 Terminology

Term	Meaning
Agent (subagent)	A Markdown file in `.claude/agents/` that defines a role: its tools, model, permission mode and preloaded skills. It runs in its own context window.
Skill	A folder in `.claude/skills/<name>/` with a `SKILL.md` entry point plus optional `references/` and `scripts/`. It describes a procedure.
MCP server	An external tool provider (Azure DevOps, Postgres, Swagger, Playwright, Chrome DevTools) connected through the Model Context Protocol.
PBI	Product Backlog Item / User Story in Azure DevOps. The unit every QA agent works on.
Hand-off	The files an agent leaves in `archive/<PBI_ID>/` for the next agent.

2. The .claude folder

Where agents and skills live, which files support them, and how the five QA agents pass work to each other.

2.1 Location

The QA agents are scoped to the automation project, so their configuration lives in `automation/.claude`, not at the repository root. Claude Code discovers project agents, skills and `.mcp.json` from the folder it is started in.

> **Start Claude Code from automation/**
> Run `claude` inside `automation/` so the QA agents, skills and MCP servers are loaded. The application's `CLAUDE.md` in the parent folder is still read, which is why the skills say “CLAUDE.md is already in your context — do not Read it again”.

2.2 Directory tree

```
automation/
├── .claude/
│   ├── agents/                          # one .md file per agent
│   │   ├── qa-planner.md
│   │   ├── qa-explorer.md
│   │   ├── qa-test-case-writer.md
│   │   ├── qa-bug-reporter.md
│   │   └── qa-automator.md
│   └── skills/                          # one folder per skill
│       ├── qa-planner/
│       │   ├── SKILL.md                 # entry point, always loaded first
│       │   ├── references/              # loaded only by the step that needs them
│       │   │   ├── ado-qa-task.md
│       │   │   ├── example.md
│       │   │   ├── feature-map.md
│       │   │   └── test-data-constraints.md
│       │   └── scripts/
│       │       └── check-feature-map.mjs
│       ├── qa-explorer/         (SKILL.md + references/)
│       ├── qa-test-case-writer/ (SKILL.md + references/)
│       ├── qa-bug-reporter/     (SKILL.md + references/)
│       └── qa-automator/        (SKILL.md + references/)
├── .mcp.json                            # project MCP servers
├── lessons/<agent>-lessons.md           # rules learned from human corrections
├── archive/<PBI_ID>/                    # agent outputs (git-ignored)
├── .secrets/qa.env                      # QA credentials (git-ignored)
├── tools/qa-login.mjs                   # signs in without exposing secrets
└── fe_automation/                       # Playwright + TypeScript project
```

2.3 agents/

Each agent file contains only YAML frontmatter: name, description, tool allowlist, model, effort, permission mode and the skill it preloads. It decides **what the agent may use**. The body (system prompt) is empty in this repository because the procedure lives in the skill. See chapter 3.

2.4 skills/

Each skill folder holds the playbook. `SKILL.md` is short (about 120–200 lines) and points to `references/` for detail, and to `scripts/` for checks that should be deterministic rather than “checked by reading”. Skills may share references: `qa-explorer` and `qa-automator` both reuse `../qa-planner/references/feature-map.md`.

2.5 Supporting files outside .claude

File / folder	Purpose	Rule
`.mcp.json`	Project MCP servers: postgres, swagger, azure-devops, chrome-devtools-mcp, playwright	Committed and shared; secrets through `${ENV_VAR}`
`lessons/<agent>-lessons.md`	Rules learned from human corrections, read at step 0 of every run	Appended only when a human corrects the agent
`archive/<PBI_ID>/`	`QA_PLAN.md`, `TEST_SCENARIOS.md`, `bugs/BUG-NN-*.md`: the hand-off between agents	Git-ignored; reviewed by a human
`.secrets/qa.env`	QA account credentials used by the sign-in helper	Git-ignored; agents never read it
`tools/qa-login.mjs`	Signs a QA account in to the QA Chrome; prints a one-line result, never the password or token	The only Bash command most agents may run
`skills/*/scripts/*.mjs`	Deterministic checks, e.g. `check-feature-map.mjs` (exit 0 in sync, 1 stale, 2 unreadable)	Prefer a script to a manual check
`fe_automation/`	Playwright project with page objects, test data and SQL queries	Only `qa-automator` writes here

2.6 The QA agent pipeline

The five agents form a pipeline around one PBI. A human review sits between exploration and the agents that write to Azure DevOps.

Stage	Agent	Command	Produces
1	`qa-planner`	`Plan Story <ID>`	`archive/<ID>/QA_PLAN.md` and the PBI's “QA Plan” task (closed)
2	`qa-explorer`	`Explore Story <ID>`	`TEST_SCENARIOS.md`, `bugs/BUG-NN-*.md` (all `PENDING`), “QA Exploration” task
3	Human	edit `Status:` lines	`CONFIRMED` / `ACCEPTED` or `REJECTED` per bug and proposed test case
4a	`qa-test-case-writer`	`Write test cases <ID>`	Test Case work items linked to the PBI (Tests / Tested By)
4b	`qa-bug-reporter`	`Report bugs <ID>`	Bug work items, children of the PBI, in its iteration
5	`qa-automator`	`Automate PBI#<ID>`	Playwright tests; Test Case tag “Should be automated” → “Automated”

Rules that make the pipeline work

- **One shared workspace.** Every agent reads and writes `archive/<PBI_ID>/`, keyed by the PBI id even when started on a Task.
- **Stable ids.** Scenarios are `FE1…` / `BE1…` and are never renumbered; downstream agents match by id, not by wording.
- **Human gate.** Only items whose status is `CONFIRMED` or `ACCEPTED` move on.
- **Idempotency.** Created ids are written back (`ado_bug:`, `Created:`) and a duplicate check runs before every create, so a re-run never duplicates work.
- **Board tracking.** Each agent tracks its run in its own child Task of the PBI (QA Plan, QA Exploration, QA Automation).

3. Agents and skills

An agent definition grants capabilities. A skill describes the procedure. Keep the two separate.

3.1 Anatomy of an agent file

```
# automation/.claude/agents/qa-planner.md
---
name: qa-planner
description: This agent will analyze the requirements, acceptance criteria of a
  certain PBI from Azure DevOps, create a plan for the following agents, and track
  it in the PBI's "QA Plan" task (create if missing, in progress, plan in
  Description, closed).
tools:
  - Read
  - Write
  - Edit
  - Grep
  - Skill
  - mcp__azure-devops
  - mcp__swagger
  - mcp__postgres
model: opus
effort: high
permissionMode: acceptEdits
skills:
 - qa-planner
---
```

Field	What it does	Guidance
`name`	Identifier used by `@agent-<name>` and by delegation	Same as the skill folder name
`description`	Tells Claude when to delegate to this agent	State what it reads, what it writes and where it tracks work
`tools`	Allowlist of tools. If omitted, the agent inherits every tool of the session	List only what the skill needs
`mcp__<server>`	Grants every tool of one MCP server	Use `mcp__<server>__<tool>` for a single tool (chapter 7)
`model`, `effort`	Model alias and reasoning effort for this agent	All QA agents use `opus` / `high`
`permissionMode`	How permission prompts behave inside the agent	`acceptEdits`: file edits run without a prompt
`skills`	Skills preloaded into the agent's context	One skill per agent in this repository
Body (optional)	System prompt of the agent	Left empty here: the skill holds the procedure

3.2 Agent or skill?

	Agent	Skill
Location	`.claude/agents/<name>.md`	`.claude/skills/<name>/SKILL.md`
Context	Runs in its own context window and returns a report	Loaded into the current session or agent on demand
Limits	Hard: tool allowlist, model, permission mode	Soft: rules the model is instructed to follow
How it is called	`@agent-<name> <task>` or automatic delegation	`/<name> <args>` or picked by its description
Size	Short: frontmatter and optional prompt	Long: workflow, guardrails, references, scripts
Use it for	A role with its own tool set (planner, explorer, automator)	A repeatable procedure that several agents or sessions may need

In this repository every agent is paired with a skill of the same name: the agent restricts the tools, the skill explains the work.

4. How to write a skill

The common elements extracted from the five QA skills. Follow this structure for every new skill.

4.1 The common skeleton

All five `SKILL.md` files follow the same ten-part structure. The order matters: the model reads top-down, so context and limits come before the procedure.

#	Section	Purpose	Example from this repository
1	Frontmatter	`name` and a `description` that states inputs, outputs and tracking	qa-automator: tag, folder, task, all in one sentence
2	Title and role	One paragraph placing the agent in the pipeline	“The step after `qa-test-case-writer`…”
3	System under test	URLs, tracker, and what is already in context	API :5080, portals :5173 / :5174, DemoApp
4	References	Table: file → step that loads it	`references/ado-qa-task.md` — Steps 0 and 11
5	Inputs	Exact commands and variants	`Report bugs <ID> BUG-02 dry run`
6	Tools / MCP servers	Read scope, write scope and a negative list per tool	azure-devops: write only the QA task
7	Workflow	Numbered steps 0…N with stop conditions	qa-explorer: 0 Board … 11 Close on the board
8	Guardrails	Rules that override everything else	Credentials, synthetic data, black-box
9	Lessons	Read at start; append on human correction	`lessons/qa-*-lessons.md`
10	Example	One realistic run, end to end	“User: Report bugs 292.” → 2 created, 1 skipped

4.2 Frontmatter and description

The description is how Claude decides to use the skill, and it is the contract the human reads first. Write it as one dense sentence that answers four questions:

1. **What does it read?** e.g. “the Test Cases of a PBI tagged *Should be automated*”.
2. **What does it produce?** e.g. “one Playwright test per Test Case in `automation/fe_automation`”.
3. **What rules matter most?** e.g. “creates and removes all its own test data”.
4. **Where is the work tracked?** e.g. “the PBI's *QA Automation* task (created if missing, in progress, summary, closed)”.

Keep `name` identical in the skill frontmatter, the skill folder and the agent file.

4.3 Role and system context

- Open with one paragraph that says **where the agent sits** in the pipeline and what it does *not* do (e.g. “It files no bugs or test cases in the tracker”).
- Give the **system under test** in a few lines or a table: URLs, the tracker organisation and project, the Epic and Feature codes.
- Name the **sources of truth** and which one wins on conflict: “when the AC and a doc disagree, the AC wins”.
- Say what is **already in context** so the agent does not waste tokens: “CLAUDE.md is already in your context — do not Read it again”.
- For large files, prescribe the access pattern: “`docs/api.md` is ~10k tokens: grep the `##` heading and Read with `offset`/`limit`”.

4.4 References, scripts and progressive disclosure

`SKILL.md` stays short; details move into `references/`. Each reference is tied to **the step that needs it**, so the agent loads it only at that moment.

From qa-explorer/SKILL.md

```
## References (load only when the step needs it)

| File                                          | When                                  |
|-----------------------------------------------|---------------------------------------|
| [references/ado-qa-task.md](...)              | Steps 0 and 11 - resolve PBI, task    |
| [references/environment.md](...)              | Step 1 - URLs, sessions, DB rules     |
| [references/fixtures.md](...)                 | Step 4 - run id, fixture recipes      |
| [references/output-templates.md](...)         | Steps 7-9 - TEST_SCENARIOS, bug file  |
| [../qa-planner/references/feature-map.md]()   | Step 3 - Feature -> page doc, routes  |
```

- **Templates** for outputs (plan, bug file, test case) live in references so the format never drifts between runs.
- **Share instead of copying:** link to another skill's reference with a relative path.
- **Scripts** in `scripts/` handle checks that must be exact. `check-feature-map.mjs` verifies that every doc and `api.md` section named in the feature map still exists.
- **Helpers** in `tools/` keep secrets away from the model: `qa-login.mjs` reads `.secrets/qa.env` and signs in through the DevTools protocol.

4.5 Inputs

Define the exact commands the skill accepts. Use the same grammar for every agent: a verb followed by a work item id.

Agent	Commands
qa-planner	`Plan Story <ID>` · `Plan Task <ID>`
qa-explorer	`Explore Story <ID>` · `Explore Task <ID>`
qa-test-case-writer	`Write test cases <ID>` · `… FE1 FE7` · `… dry run`
qa-bug-reporter	`Report bugs <ID>` · `… BUG-02 BUG-05` · `… dry run`
qa-automator	`Automate PBI#<ID>` · `… TC <id> <id>` · `… dry run`

- **Subset:** a list of ids limits the run, still respecting the status gate.
- **Dry run:** read and plan only; show what would be written; no board changes.

- **Task id accepted:** the agent resolves the parent PBI and says so.
- **No credentials as input:** the agent signs in itself through the helper; on a setup problem it asks a human.

4.6 Tools and MCP servers

For every tool, state what it may read, what it may write and what is out of bounds. End with an explicit negative list.

From qa-automator/SKILL.md

```
## Tools / MCP servers
- `azure-devops` - read the PBI, its Test Cases (`Tested By` links) and the parent
  Feature. Write **only**: the PBI's QA Automation task (create, state, Description,
  comment) and the `System.Tags` field of the Test Cases you automated in this run.
  Nothing else.
- `postgres` - read-only. Table schemas before writing a query, and the leftover
  check after the run (`run_select`).
- `Bash` - only: `node tools/qa-login.mjs ...` and
  `npm --prefix fe_automation test -- <args>`.
- `Write` / `Edit` - only inside `fe_automation/` and the lessons file.
```

- Name the allowed MCP calls (`wit_work_item get`, `wit_work_item_write update`) and forbid whole families (`repo_*`, `pipelines_*`, `search_code`).
- Say what to do when a tool fails: “auth error → ask the human to reconnect (`/mcp`); never open Azure DevOps another way”.
- Say when to stop: “Stop if the azure-devops or swagger MCP is not available, and say which one”.
- For planning and exploration, enforce **black-box** testing: expected results come from Azure DevOps, Swagger, `CLAUDE.md` and `docs/`, never from source code, PRs, commits or git history.

> **Soft rules need a hard backstop**
> Rules written in a skill are instructions to the model. Anything that must never happen (reading `.secrets/`, writing to the application) should also be blocked in the agent's `tools:` list or by a `deny` rule in `settings.json` (chapter 7).

4.7 Workflow

The workflow is a numbered list, starting at 0. Each step does one thing, names the reference it needs and states its stop condition.

Step	Pattern	Example
0	Board and lessons	Resolve the PBI, find or create the agent's QA task, set it in progress, read the lessons file
1…N	One action per step	“Read the PBI's Tested By Test Cases … None → say so, comment on the task, stop”
Verify	Prove the result	Leftover check with `postgres`; tests must actually run; re-read output against the guardrails
Limit	Bounded retries	“At most 3 fix attempts per failing test, then leave it failing and report it”
End	Report and close	Summary to the human and in the task Description; close the task only if everything succeeded

- If the run stops early, leave the task in progress with a comment that says why.
- Ask the agent to **re-read its output against the guardrails** before any external write.

- Make verification explicit: *"A test that never ran proves nothing."*

4.8 Guardrails

Guardrails come last in the procedure but override everything above them, including lessons. All five skills repeat the same families:

Guardrail	Rule
Single write target	Each agent may create or change exactly one kind of work item; everything else is read-only.
Human gate	Only `CONFIRMED` / `ACCEPTED` items are acted on; never `PENDING`, `REJECTED` or unclear; never change a decision.
Credentials	Never read `.secrets/` or `.env`, never print tokens or password hashes; invitation codes only as their last 4 characters.
Synthetic data	Run-tagged `@example.test` / `example.com` data only; never touch seeded, demo or other runs' records.
Transcribe, don't invent	Every fact comes from the source file or work item; gaps become Open Questions, never guesses.
Hands off the application	Never edit `backend/`, `frontend/`, `docs/`, `CLAUDE.md`, `AGENTS.md`; never reset the stack; never commit or push.

4.9 Hand-offs between agents

- Write outputs to `archive/<PBI_ID>/` in a format defined in a reference template.
- Add a `### Handoff` section for the next agent: what to sign in to, what to verify first, the run order.
- Use flags with exact spelling (`automate · regression · testcase`) because downstream agents parse them.
- Give every item a stable id and never renumber it.
- Write created ids back into the source file and run a duplicate check in the tracker before every create.

4.10 Lessons

Every skill reads its lessons file at step 0 and appends to it only when a human corrects it. A lesson overrides the general guidance of the skill, never its guardrails.

```
# lessons/qa-planner-lessons.md
## YYYY-MM-DD - <short title>
- Lesson: <the rule>
- Why: <what went wrong or what the user said>
- How to apply: <when and how it changes the plan>
```

Do not let the agent add lessons it inferred itself; that keeps the file small and trustworthy.

4.11 Example

Finish with one realistic run: the input, what the agent finds, what it writes and the final comment. For example, qa-bug-reporter: *“User: Report bugs 292.”* → three bug files, two `CONFIRMED`, one `REJECTED` → two Bugs created in `DemoApp\Sprint 3` as children of PBI 292, ids written back, one skipped.

4.12 Template

Copy into .claude/skills/my-skill/SKILL.md

```
---
name: my-skill
description: <What it reads>, <what it produces>, <the key rule>, and where it
  tracks the work (<task name>, created if missing, in progress, closed).
---

# My Skill

<Role: the step after X and before Y. What it does not do.>

**System under test:** <URLs>. **Tracker:** <org / project>.
CLAUDE.md is already in your context - do not Read it again.

## References (load only when the step needs it)
| File                        | When      |
|-----------------------------|-----------|
| references/<file>.md        | Step <n>  |

## Inputs
- `<Verb> <ID>` - <default scope>
- `<Verb> <ID> <ids>` - only those items
- `<Verb> <ID> dry run` - read and plan only, no writes

## Tools / MCP servers
- `<server>` - read: <...>. Write ONLY: <...>.
- Nothing else: no <tool families>. Auth error -> ask the human to run /mcp.

## Workflow
0. Lessons and board: read lessons, resolve the PBI, task in progress.
1. <step> (references/<file>.md). None found -> say so, stop.
2. <step>
N. Report to the human; write the summary; close the task.

## Guardrails
- The only thing you may create or change is <...>.
- No credentials; synthetic, run-tagged data only.
- Never edit the application; never commit.

## Lessons
Append to lessons/my-skill-lessons.md only when a human corrects you.

## Example
User: "<Verb> 123." -> <what it finds, writes and reports>
```

4.13 Checklist

1. The description says what the skill reads, produces and tracks, and matches the agent's description.
2. `SKILL.md` stays under about 200 lines; detail lives in `references/`, each tied to a step.
3. Inputs are exact commands with subset and dry-run variants.
4. Every tool has a read scope, a write scope and a negative list.
5. The workflow is numbered, has stop conditions, a verification step and bounded retries.
6. Guardrails cover the single write target, credentials, data and the human gate.
7. A lessons file exists with its format, and the skill ends with one realistic example.
8. The agent's `tools:` list contains only what the skill needs.

5. Adding MCP servers

MCP servers give agents access to Azure DevOps, the database, the API contract and the browsers.

5.1 Project servers in .mcp.json

Project-scoped servers are declared in `.mcp.json` in the folder Claude Code is started from, and are shared with the team through git. This repository has one at the root (Azure DevOps) and one in `automation/` with five servers.

```
// automation/.mcp.json (shortened)
{
  "mcpServers": {
    "azure-devops": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@azure-devops/mcp", "demo-app-gabi", "--authentication", "pat"],
      "env": { "PERSONAL_ACCESS_TOKEN": "${AZURE_DEVOPS_MCP_TOKEN}" }
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    },
    "chrome-devtools-mcp": {
      "type": "stdio",
      "command": "cmd",
      "args": ["/c", "npx", "-y", "chrome-devtools-mcp@latest",
               "--browserUrl", "http://127.0.0.1:9222"]
    }
  }
}
```

- **Secrets** use `${VAR}` expansion: the Azure DevOps PAT comes from the `AZURE_DEVOPS_MCP_TOKEN` environment variable, never from the file.
- **Windows:** wrap `npx` with `"command": "cmd"` and `"args": ["/c", "npx", …]`, as `chrome-devtools-mcp` does.
- **Approval:** the first time, Claude Code asks you to approve the project's servers.
- **Tool names** become `mcp__<server>__<tool>`, for example `mcp__azure-devops__wit_work_item`. These are the names used in agent `tools:` lists and permission rules.

5.2 The claude mcp CLI

```
# stdio server, shared with the team (written to .mcp.json)
claude mcp add --scope project playwright -- npx -y @playwright/mcp@latest

# stdio server with an environment variable
claude mcp add --scope project azure-devops \
  -e PERSONAL_ACCESS_TOKEN='${AZURE_DEVOPS_MCP_TOKEN}' \
  -- npx -y @azure-devops/mcp demo-app-gabi --authentication pat

# remote HTTP server
claude mcp add --transport http <name> <https-url>

# manage servers
claude mcp list              # all servers and their status
claude mcp get playwright    # details of one server
claude mcp remove playwright
```

Everything after `--` is the command that starts the server.

5.3 Scopes

Scope	Stored in	Visible to	Use it for
`local` (default)	`~/.claude.json`	Only you, in this project	Personal or experimental servers
`project`	`.mcp.json` in the project	Everyone who clones the repository	The QA servers every agent needs
`user`	`~/.claude.json`	Only you, in all projects	Tools you use everywhere

5.4 Verifying a server

- Inside a session, run `/mcp` to see each server's status, its tools, and to re-authenticate.
- The skills rely on this: “If the tool fails with an auth error, ask the human to reconnect (`/mcp`)”.
- Servers that need the application (`swagger`, `postgres`) only work while the stack is running.

6. Calling an agent

Use an @-mention to run a specific agent with a task written in its skill's input format.

6.1 Syntax

```
@agent-qa-planner Plan Story 112
@agent-qa-explorer Explore Story 112
@agent-qa-test-case-writer Write test cases 112 dry run
@agent-qa-bug-reporter Report bugs 112 BUG-01 BUG-03
@agent-qa-automator Automate PBI#50
```

- Type `@` in the prompt and pick the agent from the list; agents appear as `agent-<name>`.
- Follow it with the task, using the exact command from the skill's **Inputs** section.
- The mention guarantees that this agent runs. It works in its own context and returns a report to the main session.
- Natural language also works (“Use the qa-planner agent to plan Story 112”), but the mention is explicit and repeatable.

6.2 Related commands

Command	What it does
`@agent-<name> <task>`	Runs that agent with the task
`/agents`	Lists, creates and edits agents, including their tool access
`/<skill-name> <args>`	Runs a skill in the current session, e.g. `/qa-planner Plan Story 112`
`/mcp`	Checks that the MCP servers the agent needs are connected
`/permissions`	Shows and edits the permission rules in effect

> **Running the pipeline**
> Run the agents one at a time along the pipeline (section 2.6). Review the `Status:` lines in `archive/<PBI_ID>/` between `qa-explorer` and the writer / reporter agents; they act only on what a human accepted.

7. Permissions

How this repository restricts what agents can touch today, and how to enforce it with settings.json.

7.1 How this repository does it today

Access is currently controlled in four layers. Only the first two are enforced by Claude Code; the others rely on the model or on keeping files out of reach.

Layer	Where	How it is used here	Enforced?
Tool allowlist	`tools:` in the agent file	qa-bug-reporter and qa-test-case-writer get Read, Glob, Grep, Edit, Skill and `mcp__azure-devops` — no Write, no Bash, no browser, no database	Yes
Permission mode	`permissionMode:` in the agent file	All five agents use `acceptEdits`: file edits run without a prompt; Bash and MCP calls follow the normal rules	Yes
Skill rules	Tools and Guardrails sections of `SKILL.md`	"Write/Edit only under `archive/<PBI_ID>/`", "Bash only `node tools/qa-login.mjs`", "azure-devops: write only the QA task"	No (instructions)
Out of reach	`.gitignore`, `tools/qa-login.mjs`	`archive/` and `.secrets/` are not committed; the login helper reads the secrets so the model never has to	Partly

MCP granularity

Entry	Grants
`mcp__azure-devops`	Every tool of the server, including writes to repositories, pipelines and wikis
`mcp__azure-devops__wit_work_item`	Only that one tool (read work items)

All agents in this repository grant whole servers and narrow them in prose. Section 7.2 shows how to narrow them in configuration instead.

Permission modes

Mode	Behaviour
`default`	Prompts for each tool the first time it is used
`acceptEdits`	Accepts file edits automatically; other tools still follow the rules
`plan`	Read-only planning; no edits or commands
`bypassPermissions`	Skips all prompts. Avoid for agents that write to external systems

7.2 Rules in settings.json

Permission rules live in a `permissions` object with three lists. They apply to the main session and to every agent, whatever its permission mode.

```
// automation/.claude/settings.json  (proposal)
{
  "permissions": {
    "allow": [
      "Bash(node tools/qa-login.mjs:*)",
      "Bash(npm --prefix fe_automation test:*)",
      "Read(../docs/**)",
      "mcp__swagger",
      "mcp__azure-devops__wit_work_item"
    ],
    "ask": [
      "mcp__azure-devops__wit_work_item_write"
    ],
    "deny": [
      "Read(./.secrets/**)",
      "Read(./fe_automation/.env)",
      "Edit(../backend/**)",
      "Edit(../frontend/**)",
      "mcp__azure-devops__repo_create_branch",
      "mcp__azure-devops__pipelines_run"
    ]
  },
  "enabledMcpjsonServers": ["azure-devops", "swagger", "postgres"]
}
```

Rule type	Syntax	Example
Files (read)	`Read(<glob>)`	`Read(./.secrets/**)`
Files (write)	`Edit(<glob>)`	`Edit(../backend/**)`
Shell	`Bash(<command prefix>:*)`	`Bash(npm --prefix fe_automation test:*)`
MCP, whole server	`mcp__<server>`	`mcp__swagger`
MCP, one tool	`mcp__<server>__<tool>`	`mcp__azure-devops__wit_work_item_write`

- **Precedence:** `deny` beats `ask`, which beats `allow`. A deny rule blocks the action even for an agent in `acceptEdits` mode.
- **Globs** follow gitignore syntax: `**` matches any depth.
- **Project servers** can be enabled one by one with `enabledMcpjsonServers` instead of approving all of them.

7.3 Where settings live

File	Scope	Committed?
`.claude/settings.json`	Team rules for this project	Yes
`.claude/settings.local.json`	Your personal overrides for this project	No (git-ignored)
`~/.claude/settings.json`	Your rules for every project	No

Use `/permissions` to inspect the rules in effect and to add or remove them interactively.

8. Recommendations

Gaps found in the current configuration while writing this guide.

#	Finding	Recommendation
1	The skills forbid reading `.secrets/` and `fe_automation/.env`, but no `settings.json` exists, so nothing enforces it.	Add `automation/.claude/settings.json` with `deny` rules for secrets and for edits to the application (section 7.2).
2	Every agent is granted the whole `mcp__azure-devops` server, including repository, pipeline and wiki writes.	List only the tools each skill names, e.g. `mcp__azure-devops__wit_work_item` and `mcp__azure-devops__wit_work_item_write`.
3	`automation/.mcp.json` contains the database URL inline and the swagger / postgres servers use absolute `C:/Users/...` paths.	Use `${DATABASE_URL}` and repository-relative or environment-based paths so the file works on every machine.
4	Exceptions to a whole-server grant are written only in prose.	Use `disallowedTools` in the agent frontmatter, or `deny` rules, to remove risky tools while keeping the rest of the server.

Appendix A. Agent quick reference

Agent	Command	Tools (agent file)	Writes locally	Writes in Azure DevOps
qa-planner	`Plan Story <ID>`	Read, Write, Edit, Grep, Skill, azure-devops, swagger, postgres	`archive/<ID>/QA_PLAN.md`, lessons	Its “QA Plan” task
qa-explorer	`Explore Story <ID>`	Read, Write, Edit, Glob, Grep, Skill, Bash, chrome-devtools-mcp, azure-devops, postgres, swagger	`TEST_SCENARIOS.md`, `bugs/`, lessons	Its “QA Exploration” task
qa-test-case-writer	`Write test cases <ID>`	Read, Glob, Grep, Edit, Skill, azure-devops	`Created:` line in accepted blocks, lessons	Test Cases + Tests links
qa-bug-reporter	`Report bugs <ID>`	Read, Glob, Grep, Edit, Skill, azure-devops	`ado_bug` write-back in bug files, lessons	Bugs + parent / related links
qa-automator	`Automate PBI#<ID>`	Read, Write, Edit, Glob, Grep, Skill, Bash, azure-devops, playwright, chrome-devtools-mcp, postgres, swagger	`fe_automation/` specs, page objects, data, queries; lessons	Its “QA Automation” task + Test Case tags

All agents use `model: opus`, `effort: high` and `permissionMode: acceptEdits`, and preload the skill of the same name.